Programming Languages and Tools

Languages Used

Language	Where it is used	Why it is used	Advantages in this system
C#	All main projects	Core application, game logic, display code, tests, and HTTP clients	Strong typing, rich standard library, good .NET ecosystem support, and straightforward UI/hardware integration.
Markdown	Documentation files and existing `README.md` / `uml.md`	Human-readable technical documentation	Easy to version control and review, and supports Mermaid diagrams.
JSON	`appsettings.json`, `shared/latest_score.json`, tool manifests	Configuration and data exchange	Lightweight, easy to parse, and compatible with .NET configuration and HTTP payloads.
Mermaid syntax	This evidence pack and `uml.md`	Modelling and architecture diagrams	Keeps diagrams close to the documentation source and easy to maintain.

Frameworks and Libraries

Technology	Used in	Why it is used	Value to the project
.NET 9	`ConsoleTest`, `PixelBoardDisplay`, `PixelCatsClient`, `ConsoleTest.Tests`	Main runtime for console, library, and test code	Modern runtime with strong async, HTTP, and configuration support.
.NET 8	`Client/`	MonoGame desktop client target	Keeps the client project aligned with its graphics/runtime requirements.
MonoGame DesktopGL	`Client/` and `HerdingCats/`	Game rendering and desktop window hosting	Suitable for 2D game UIs and cross-platform desktop graphics.
Microsoft.Extensions.Configuration	`ConsoleTest/Program.cs`, `ConsoleTest.csproj`	Load appsettings and environment variables	Centralizes configuration without hard-coding deployment values.
System.Text.Json / HttpClient.Json	`PixelCatsClient/`, `ConsoleTest/`	JSON serialization and HTTP	Native .NET support with low overhead and

Technology	Used in	Why it is used	Value to the project
		request handling	simple DTO mapping.
System.IO.Ports	`PixelBoardDisplay/SerialPortManager.cs`	Serial communication with hardware	Direct hardware integration for the Arduino display path.
xUnit	`ConsoleTest.Tests/`	Unit testing framework	Simple, lightweight, and well-supported for .NET projects.
coverlet.collector	`ConsoleTest.Tests/`	Code coverage collection	Helps measure test depth when running the suite.

Software Tools

Tool	Evidence	Why it is used	Advantages
Visual Studio solution file	`PixelGame.sln`	Organizes the multi-project workspace	Makes the project easy to load, build, and navigate.
.NET CLI	README and project files	Build, run, test, and restore	Standard toolchain for .NET automation.
MGCB / MonoGame content builder	`.config/dotnet-tools.json`, `Client/Client.csproj`	Builds MonoGame content pipelines	Required for game assets and desktop graphics projects.
PowerShell	`tools/GenerateTestCaseReport.ps1`	Documentation/report generation support	Fits the Windows-first environment and repo tooling.
Visual Studio / VS Code	Solution and folder structure	Primary IDE assumptions	Supports C#, markdown, JSON, and project-aware navigation.

Build System

- `PixelGame.sln` ties the projects together.
- Each project uses an SDK-style `.csproj` file.
- `ConsoleTest/ConsoleTest.csproj` references `PixelBoardDisplay/PixelBoard.csproj` and `PixelCatsClient/PixelCatsClient.csproj`.
- `ConsoleTest.Tests/ConsoleTest.Tests.csproj` references the application projects so tests can exercise real code paths.
- `Client/Client.csproj` uses MonoGame content tooling and Windows desktop settings.

Testing Tools

- xUnit test classes in `ConsoleTest.Tests/` cover game lifecycles, API client behavior, and atomic file writing.
- `FakeHttpMessageHandler` is used to isolate HTTP behavior from the network.
- Reflection-based tests validate `WriteScoreFileAtomic` without changing application logic.

Deployment Tooling

- The runtime is deployed as .NET desktop/console executables rather than as a hosted service.
- `ConsoleTest` can be run directly with `dotnet run --project ConsoleTest/ConsoleTest.csproj`.
- The MonoGame projects rely on the content pipeline toolchain and desktop runtime support.
- Hardware deployment assumes a Windows machine with serial access to the configured COM port.

IDE and Toolchain Assumptions

- A .NET-aware IDE is assumed for solution navigation and debugging.
- Windows support is assumed for the hardware paths because the display/input layer uses Windows-specific APIs.
- The repository already contains `.config/dotnet-tools.json` manifests, which indicates source-controlled toolchain state.